Intelligent-Resource-Monitoring-Auto-Scaling-Pipeline

Gourav Garg | M25AI2164

Github Repo Link: https://github.com/ggarg55/Intelligent-Resource-Monitoring-Auto-Scaling-Pipeline

---

1. Introduction

In this project, I built a simple system that can **automatically scale resources** based on traffic. The main idea is to monitor how much load an application is receiving and create new cloud machines when the load becomes high.

---

2. Objective

The goal of this project was:

- To monitor an application running locally
- To detect high traffic using metrics
- To trigger alerts when traffic crosses a limit
- To automatically create a new virtual machine on cloud (GCP)
- To visualize system behavior using dashboards

---

3. System Overview

The system works in the following flow:

1. A Flask application receives user traffic
2. Prometheus collects metrics from the application
3. When traffic increases, an alert rule is triggered

4. Alertmanager sends this alert to a webhook
5. The webhook creates a new VM in Google Cloud
6. Grafana shows all metrics in graph form

---

4. Tools and Technologies Used

Flask (Python)

Used to create a simple backend application and expose metrics.

Prometheus

Used to collect and store metrics like number of requests.

Grafana

Used to create dashboards and visualize traffic in real-time.

Alertmanager

Used to manage and send alerts when conditions are met.

Webhook (Python Script)

A custom service that receives alerts and performs actions (like creating VM).

Google Cloud Platform (GCP)

Used to create virtual machines dynamically using the `gcloud` command.

---

5. Implementation Steps

Step 1: Local VM Setup

- Created Ubuntu VM using VirtualBox
- Installed required tools and dependencies

Step 2: Application Setup

- Built a Flask app

- Added `/metrics` endpoint using Prometheus client

Step 3: Monitoring Setup

- Installed Prometheus
- Configured it to scrape metrics from Flask app

Step 4: Alert Configuration

- Created alert rules based on request rate
- Example: high traffic in 1 minute

Step 5: Alert Routing

- Installed Alertmanager
- Connected Prometheus to Alertmanager
- Configured webhook as receiver

Step 6: Webhook Implementation

- Created a Python service
- It listens for alerts
- On alert, it runs `gcloud` command to create VM

Step 7: Visualization

- Installed Grafana
- Connected it with Prometheus
- Created dashboards to show traffic

---

6. Additional Improvements (Beyond Basic Requirements)

In this project, I also added some extra features to make it more realistic:

Cooldown Mechanism

- Prevents multiple VM creation in short time
- Avoids unnecessary cost

Lock File System

- Ensures only one scaling action runs at a time

Automation Script

- Created `start-all.sh` to start all services with one command
- No need to open multiple terminals

Dashboard Visualization

- Used Grafana to clearly show traffic spikes
- Makes the system easier to understand

End-to-End Testing

- Used load testing tool (`ab`) to simulate real traffic
- Verified full pipeline: monitoring → alert → scaling

---

7. Results

- The system successfully detected high traffic
- Alerts were triggered correctly
- Webhook created new VM in GCP automatically
- Grafana showed traffic spike clearly

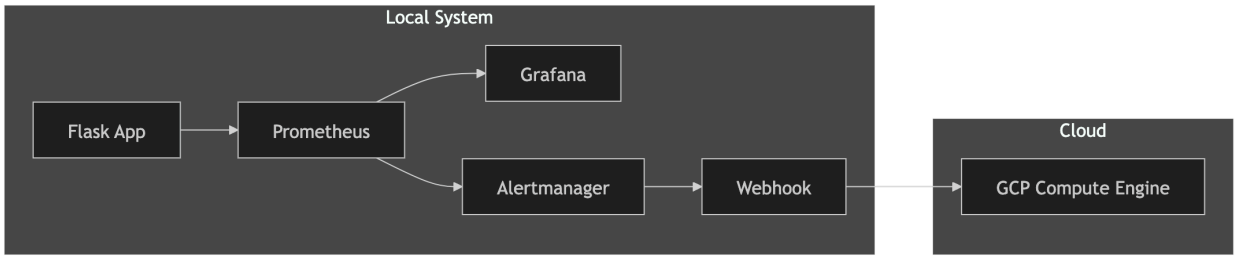
This proves that the system can **scale automatically based on demand**.

---

8. Challenges Faced

- Setting up Prometheus and alert rules correctly
- Managing file paths and configurations
- Debugging alerts and webhook issues

9. Archiecture



10. Conclusion

By extending the original assignment, this project now demonstrates a complete real-world workflow instead of just individual concepts. It shows how a system can continuously monitor itself, detect unusual traffic patterns, and automatically respond by provisioning cloud resources.

Through this implementation, the project clearly highlights:

- How monitoring systems collect and analyze real-time data
- How alerts are triggered and routed based on defined conditions
- How cloud infrastructure can be created automatically without manual effort
- How automation helps build scalable and reliable systems